# Task1: Backend - Identity Reconciliation

## Task Overview:

Dr. Chandrashekar, affectionately known as Doc, is currently stranded in the year 2023, engrossed in fixing his time machine to embark on a journey back to the past and save a dear friend. As an avid patron of the popular online store Zamazon.com, Doc exercises great caution by using different email addresses and phone numbers for each purchase to avoid unwanted attention to his ambitious project.

The company, known for its expertise in enhancing customer experiences, has decided to integrate its cutting-edge technology into Zamazon.com. This integration is designed to collect and manage contact details from shoppers, providing a personalised customer experience.

However, given Doc's unique approach to anonymity, Company faces a distinctive challenge: linking different orders made with different contact information to the same individual.

## Your Mission:

1. Develop a web service capable of processing JSON payloads with "email" and "phoneNumber" fields, creating a shadowy infrastructure that consolidates contact information across multiple purchases.
2. Craft a response mechanism that returns an HTTP 200 status code, along with a JSON payload containing consolidated contact details. The payload should be cunningly structured with a "primaryContactId," "emails," "phoneNumbers," and "secondaryContactIds."
3. Be prepared for a scenario where no existing contacts match the incoming request. In such cases, your service should craft a new entry in the database, treating it as a discreet individual with no affiliations.
4. Implement a strategy where incoming requests matching existing contacts trigger the creation of "secondary" contact entries, providing a covert mechanism for storing new information.
5. Exercise caution as primary contacts can seamlessly transform into secondary contacts if subsequent requests reveal overlapping contact information. This dual-purpose functionality adds an extra layer of complexity to your mission.

Schema reference:

```
Unset
{

      id          Int

 phoneNumbe       String
 r                ?
 email         String
               ?
 linkedI         Int? // the ID of another Contact linked to
 d               this one
 linkPrecedenc     "secondary"|"primary" // "primary" if it's
e                the
first Contact in the
link
 createdA        DateTim
 t               e
 updatedA        DateTim
 t               e
 deletedA        DateTime
 t               ?
}
```

## Requirements:

1. Meticulously implement the ***/identify*** endpoint, ensuring that it operates with utmost discretion and precision.
2. Execute the creation of a new "Contact" entry with ***linkPrecedence="primary"*** when no existing contacts match the incoming request.
3. Employ a covert strategy for creating "secondary" contact entries when incoming requests match existing contacts and introduce new information.
4. Maintain the integrity of the database state, executing updates seamlessly with each incoming request.

## Bonus Points:

- Craft an error handling system that misdirects potential threats and provides misleading error responses.
- Employ covert optimization techniques for database queries and operations to operate under the radar.
- Execute a covert unit testing strategy to validate the functionality of your service without revealing its true nature.
- Anticipate and handle edge cases with the finesse of a seasoned operative.

## Submission guidelines:

1. Make the github repo public and add a readme file, with proper steps for execution.
2. FIll the form and paste the github links.
3. Record a video of the code running and explain the code.
4. Write proper comments in the code.